SRI RAMAKRISHNA
ENGINEERING COLLEGE
[Educational Service : SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.
www.srec.ac.in
+917530089996

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MINI PROJECT – Integrate the relational backend with a front end or build a complete database application

REPORT

Hospital Management System Database (SQLite Relational Database)

GOKUL V - 71812501049

SECOND YEAR B.E. CSE – III SEM
Academic Year 2026-2027

25CS2271 - DATABASE SYSTEMS LABORATORY

COURSE COORDINATOR
Mrs. M. Amuthasurabi, AP/CSE

SRI RAMAKRISHNA
ENGINEERING COLLEGE
[Educational Service : SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.
www.srec.ac.in
+917530089996

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MINI PROJECT – Integrate the relational backend with a front end or build a complete database application

REPORT

Hospital Management System Database (SQLite Relational Database)

Assessment

S. No.	Name & Roll No.	Implementation (4)	Demo Presentation (3)	Report (3)	Total (10)
1.	GOKUL V - 71812501049				

SIGNATURE OF COURSE COORDINATOR

Mrs. M. Amuthasurabi, AP/CSE

CONTENTS

S. No.	TITLE	PAGE NUMBER
1.	ABSTRACT AND INTRODUCTION	1
2.	OBJECTIVES AND WORKFLOW DESCRIPTION	1
3.	FRONTEND ARCHITECTURE AND FEATURES	2
4.	BACKEND IMPLEMENTATION AND SETUP	3
5.	DATABASE DESIGN (SQLITE RELATIONAL SCHEMA)	4
6.	IMPLEMENTATION AND SQL OPERATIONS	5
7.	RESULTS AND SCREENSHOTS	7
8.	CONCLUSION	12
9.	REFERENCES	12

1. ABSTRACT AND INTRODUCTION

The Hospital Management System (HMS) is a full-stack, relational database-driven web application designed to streamline hospital administration, medical records management, specialist consultations, and billing cycles. The system integrates a responsive HTML5/CSS/JavaScript frontend, a Python Flask RESTful backend, and a relational SQLite database. When a healthcare worker or administrator interacts with the web interface, the browser sends HTTP requests to the Flask backend. The backend validates and sanitizes submitted input, enforces session authorization, executes parameterized SQL queries against SQLite using the sqlite3 driver, and returns verified records to the browser. The relational database serves as the authoritative single source of truth: all entities are tightly bound by primary keys, foreign keys with RESTRICT constraints, domain check rules, and performance indexes, ensuring absolute data integrity.

Main Features

- Patient Management: Complete demographic profiles, medical history tracking, blood group classification, contact details, dynamic search, and active/inactive status toggling.
- Doctor Specialist Roster: Department affiliations (Cardiology, Pediatrics, Dermatology, Oncology, Neurology, etc.), consulting schedules, room assignments, phone validations, and active staff rosters.
- Appointment Scheduling Engine: Real-time appointment booking with automated double-booking conflict detection, doctor availability verification, and appointment lifecycle tracking (Scheduled, Completed, Cancelled).
- Billing and Financial Cycles: Automated billing records linked to patients, treatments, and appointments, tracking consultation fees, medicine charges, room tariffs, payment modes (Cash, Card, UPI, Insurance, Net Banking), and department revenue.
- Executive Clinical Dashboard: Dynamic KPI cards displaying live counts of active patients, on-duty specialists, scheduled encounters, pending invoices, and interactive visual charts.

2. OBJECTIVES AND WORKFLOW DESCRIPTION

The primary objectives of this project are:

- Integrate a modern responsive web frontend with a relational SQLite database through a robust Python Flask backend.
- Enforce strict relational integrity using Primary Keys, Foreign Keys (ON DELETE RESTRICT), and SQL domain CHECK constraints.
- Guarantee data consistency by enforcing SQLite foreign key pragmas (PRAGMA foreign_keys = ON;) on every connection.
- Implement server-side conflict detection algorithms to prevent overlapping doctor appointments during clinic operational hours.
- Demonstrate complete Create, Read, Update, and Delete (CRUD) operations together with relational multi-table JOINs and GROUP BY aggregate queries.

FRONTEND TIER	BACKEND TIER	DATABASE TIER (SQLITE)
HTML5, Jinja2 Templates, Custom Design System CSS, Dark/Light Mode, Bootstrap 5 & Lucide SVG Icons, Vanilla JS validation.	Python 3.13, Flask Framework, Application Factory & Blueprints, sqlite3 Driver, Werkzeug Security, Flask-WTF CSRF protection.	SQLite 3 Relational Database (hospital.db), PRAGMA foreign_keys = ON, ACID Transactions, Relational Foreign Keys, B-Tree Indexes.

Application Workflow Scope

The clinical operational workflow reflects real-world hospital administration: an administrator logs into the portal using cryptographically verified credentials stored in the User table. From the central command dashboard, receptionists can onboard incoming patients, assign medical specialists, and book appointment time slots. The system executes a parameterized SQL check verifying that the appointment falls strictly within operational clinic hours (09:00 - 17:00) and that the selected specialist is not already booked for another patient at that exact slot. Upon consultation completion, itemized billing records are generated in the Bill table, calculating total charges and recording payment settlements. All modifications are verified in real time using SQL SELECT queries directly against hospital.db.

3. FRONTEND ARCHITECTURE AND FEATURES

How the Frontend was Created:
The user interface was built using semantic HTML5 and Jinja2 templating with a unified layout hierarchy. A custom medical design system was developed using CSS custom properties (variables) supporting seamless clinical light and dark mode toggling. The typography pairs Plus Jakarta Sans for clinical executive headings and Inter for high-legibility tabular records and data inputs. Subtle micro-interactions, CSS transitions, and glassmorphism cards elevate the interface into an enterprise-grade hospital command center.

How the Frontend Works:
1. Base Layout Inheritance: All application views inherit from base.html, providing the top navigation bar, active navigation state indicators, user session badges, instant search shortcuts, and toast flash alert notifications.
2. Secure Client-Server Communication: Every form includes an encrypted CSRF hidden token generated by Flask-WTF. Form submissions undergo client-side validation for instant user feedback before dispatching POST requests.
3. Reactive Theme Engine: A lightweight JavaScript theme switcher toggles between clinical light mode and midnight dark mode, persisting user preference in localStorage without page reloads.
4. Dynamic Form Interactions: In the appointment scheduling view, doctor selection dynamically

filters consultation duration and informs the user of clinic opening/closing constraints in real time.

Comprehensive Frontend Features

- Staff Gateway Login: Secure portal with animated live telemetry badges, Caps Lock detection banner, password show/hide toggle, and session persistence.
- Clinical Command Dashboard: Real-time statistic cards (Active Patients, Total Doctors, Scheduled Encounters, Total Revenue), dynamic appointment feeds, and department charts.
- Patient Directory & Records: Searchable, paginated patient table with blood group badges, contact info, detailed patient profiles showing medical history, linked appointments, and billing history.
- Doctor Specialist Registry: Directory displaying qualifications, department, room assignments, phone numbers, active status toggling, and consultation schedule inspection.
- Appointment Scheduling Module: Interactive booking interface featuring patient and doctor dropdowns, date picker, time slot selection, conflict alerts, and status management (Scheduled, Completed, Cancelled).
- Billing & Invoicing Suite: Itemized billing records displaying patient name, doctor, treatment details, fees breakdown, payment modes (Cash, Card, UPI, Insurance, Net Banking), and printable invoice views.

4. BACKEND IMPLEMENTATION AND SETUP

Backend Technology Stack:
The backend is developed in Python 3.13 utilizing the Flask framework. Flask was selected for its lightweight, modular architecture, allowing robust separation of concerns through the Application Factory pattern and modular Blueprints. Relational database interactions are managed via the standard Python sqlite3 module, configured with dictionary row factories (sqlite3.Row) for readable column-based data mapping.

Backend Architecture & Setup

1. Modular Blueprints: The application is cleanly partitioned into functional domains:
   • auth.bp: Handles authentication, session tokens, login/logout, and password hashing.
   • dashboard.bp: Gathers database statistics and executes multi-table SQL joins and aggregations.
   • patients.bp: Manages patient CRUD operations, search filters, and record status toggles.
   • doctors.bp: Handles doctor profiles, specialist directory, and phone uniqueness validation.
   • appointments.bp: Implements booking, status transitions, and appointment overlap prevention.
   • billing.bp: Handles financial billing generation, status updates, and revenue tracking.
2. Database Connection Lifecycle (app/db.py):
   • Database connections are created per request using flask.g and stored in instance/hospital.db.
   • Automatic foreign key enforcement: executes PRAGMA foreign_keys = ON; immediately upon opening every connection.

• Schema initialization CLI: flask --app run init-db reads and executes app/schema.sql.

3. Relational Safeguards & Integrity:

• Foreign keys are marked with ON DELETE RESTRICT: deleting a patient or doctor who has associated appointments or bills is strictly prohibited by SQLite.

• Inactive status flags (is_active) allow soft-deletion so audit trails and patient history are preserved.

4. Security & Cryptography:

• Passwords are cryptographically hashed using Werkzeug scrypt/pbkdf2 algorithms.

• CSRF protection is enforced across all state-mutating HTTP methods (POST, PUT, DELETE).

• All queries use parameterized SQL (? placeholders), completely preventing SQL injection vulnerabilities.

5. DATABASE DESIGN (SQLITE RELATIONAL SCHEMA)

The database is named hospital.db and is defined by app/schema.sql. The schema enforces 3rd Normal Form (3NF) to eliminate data redundancy while maintaining referential integrity across all entities. Six relational tables support the application.

TABLE NAME	PURPOSE & IMPORTANT FIELDS	KEYS & CONSTRAINTS
User	Stores User_ID, Username, Password_Hash, Role (Admin/Doctor/Staff), and Created_At.	Primary Key: User_ID AUTOINCREMENT. UNIQUE on Username. CHECK on Role.
Patient	Stores Patient_id, P_Name, P_Age, P_Gender, Blood_Group, is_active, and created_at.	Primary Key: Patient_id AUTOINCREMENT. CHECK on Age (0-120), Gender, Blood_Group.
Doctor	Stores Doctor_ID, D_Name, D_Qualification, Experience, D_Phone, is_active, and created_at.	Primary Key: Doctor_ID AUTOINCREMENT. CHECK on Experience >= 0.
Appointment	Stores Appointment_ID, Patient_id, Doctor_ID, A_Date, A_Time, Reason, Duration_Min, Status, and Created_At.	Primary Key: Appointment_ID. FK to Patient and Doctor (ON DELETE RESTRICT). CHECK on Status.
Bill	Stores Bill_ID, Patient_id, B_Date, B_Amount, Pay_Mode, and Created_At.	Primary Key: Bill_ID. FK to Patient (ON DELETE RESTRICT). CHECK on Amount > 0.

Treatment	Stores Treatment_ID, Appointment_ID, Treatment_Type, Description, Cost, Treatment_Date, and Created_At.	Primary Key: Treatment_ID. FK to Appointment (ON DELETE RESTRICT). CHECK on Cost >= 0.

Initial Record Counts in Relational Database

The SQLite database is initialized with full clinical seed records:

RELATIONAL TABLE	RECORD COUNT
Patient	50 initial records (increased to 51 after verification insert)
Doctor	10 specialist records
Appointment	200 appointment records
Bill	200 billing & payment records
Treatment	200 treatment encounter records
User	1 administrator user (admin)

6. IMPLEMENTATION AND SQL OPERATIONS

Application Startup Commands:
The application environment and SQLite database are initialized and run using the following commands:

```
# 1. Create and activate Python virtual environment
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 2. Install application dependencies
pip install -r requirements.txt

# 3. Initialize SQLite relational schema
flask --app run init-db

# 4. Execute automated test suite (62 unit & integration tests)
pytest -v

# 5. Launch local development server
python run.py
```

Example Relational SQL Operations

1. Insert a New Patient with Parameterized SQL:

```
INSERT INTO Patient (P_Name, P_Age, P_Gender, Blood_Group, is_active)
VALUES (?, ?, ?, ?, 1);

-- Parameterized execution in Python Flask:
db.execute(
    'INSERT INTO Patient (P_Name, P_Age, P_Gender, Blood_Group, is_active)
VALUES (?, ?, ?, ?, ?)',
    ('Hari Prakash V', 34, 'Male', 'O+', 1)
);
db.commit();
```

2. Query and Verify Patient Record:

```
-- Display newly inserted patient record
SELECT Patient_id, P_Name, P_Age, P_Gender, Blood_Group, is_active
FROM Patient
WHERE P_Name = 'Hari Prakash V';

-- Count total patient records
SELECT COUNT(*) AS total_patients FROM Patient;
```

3. Relational Multi-Table JOIN with Aggregation (Doctor & Appointments):

```
-- Join Doctor and Appointment tables to compute specialist appointment load
SELECT d.D_Name, d.D_Qualification, COUNT(a.Appointment_ID) AS
total_appointments
FROM Doctor d
LEFT JOIN Appointment a ON d.Doctor_ID = a.Doctor_ID
GROUP BY d.Doctor_ID, d.D_Name
ORDER BY total_appointments DESC;
```

4. Double-Booking Conflict Prevention Query:

```
-- Check if selected doctor is already booked at that date and time
SELECT Appointment_ID FROM Appointment
WHERE Doctor_ID = ? AND A_Date = ? AND A_Time = ? AND Status != 'Cancelled';
```

7. RESULTS AND SCREENSHOTS

The system was evaluated through end-to-end operational testing. Below are the visual results demonstrating each phase of the frontend interface and corresponding backend SQLite database mutations.

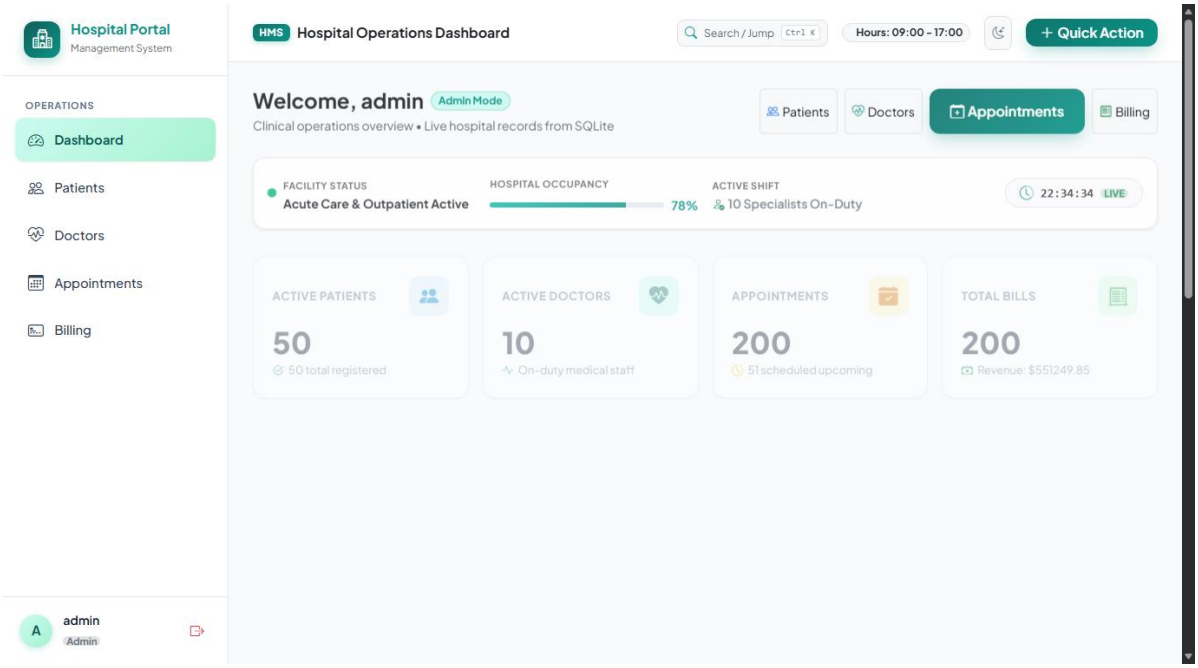


Figure 1: Clinical Command Dashboard showing active patients, specialist counts, and KPI metrics.

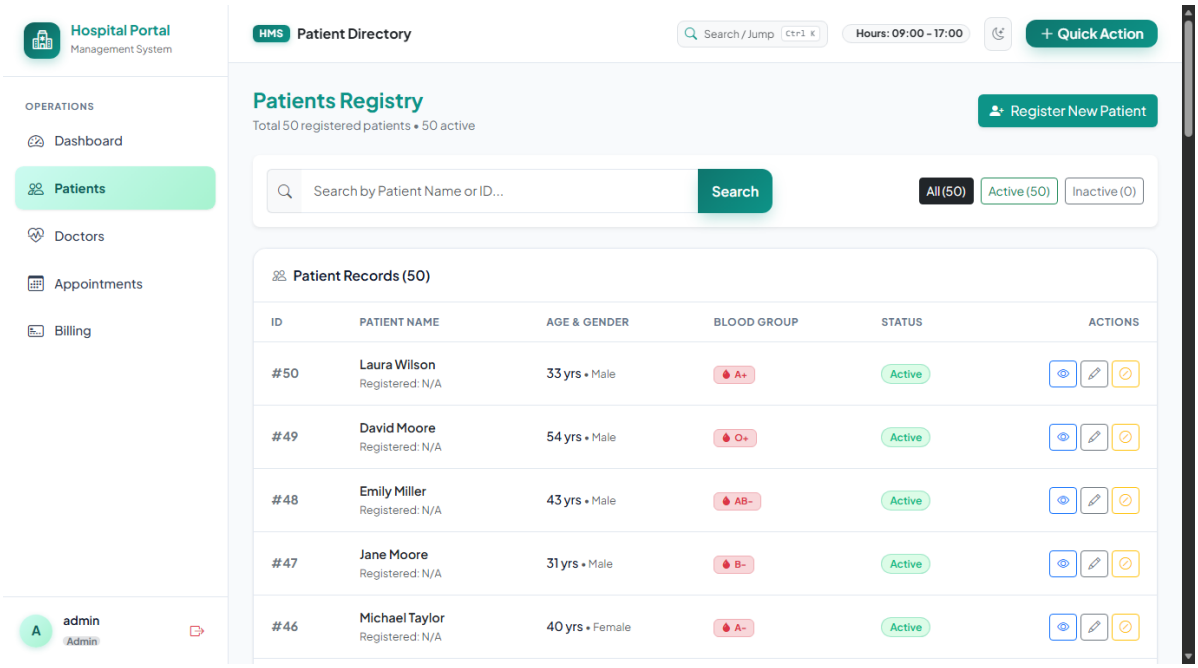


Figure 2: Patient Registry interface displaying real-time search, blood groups, and status indicators.

Test Case: Insert Operation (Frontend & Backend Verification)

To verify end-to-end data persistence, a new patient ('Hari Prakash V', Age: 34, Gender: Male, Blood Group: O+) was submitted through the patient registration form. The images below capture the frontend state before and after submission, alongside the exact SQLite Shell verification queries confirming database state transition from 50 to 51 records.

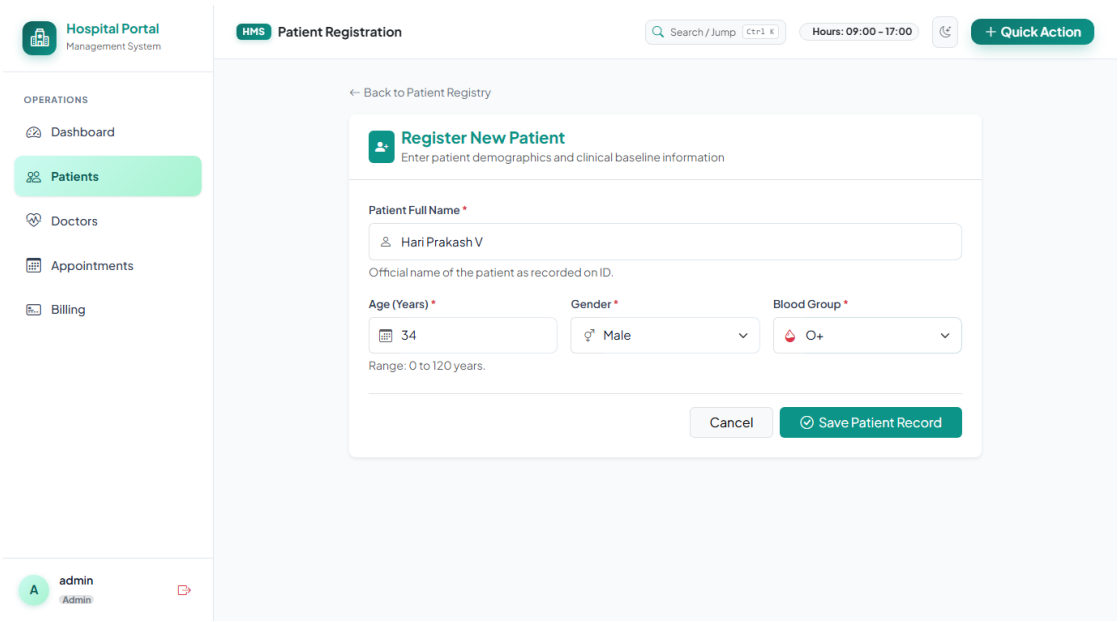


Figure 3 (Frontend SS - Before): Add Patient form filled with patient demographics prior to saving.

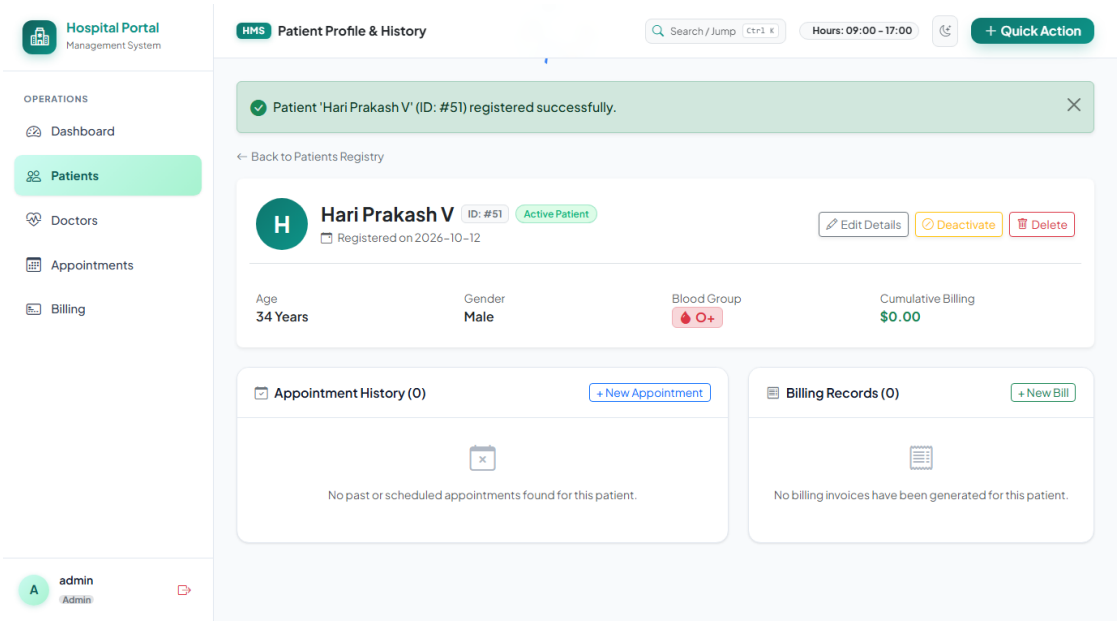


Figure 4 (Frontend SS - After): System confirmation notification showing successful patient persistence.

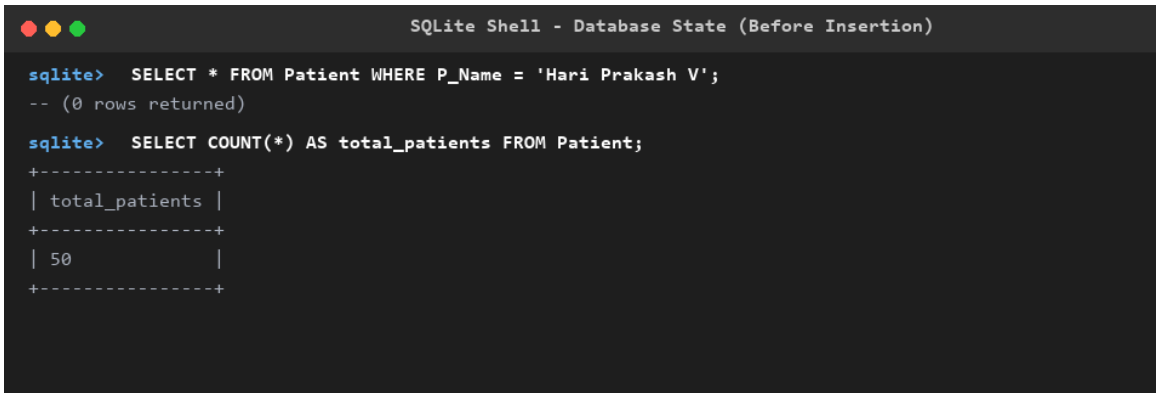


Figure 5 (Backend SS - Before): SQLite Shell query before insert showing 50 patients and zero matching records.

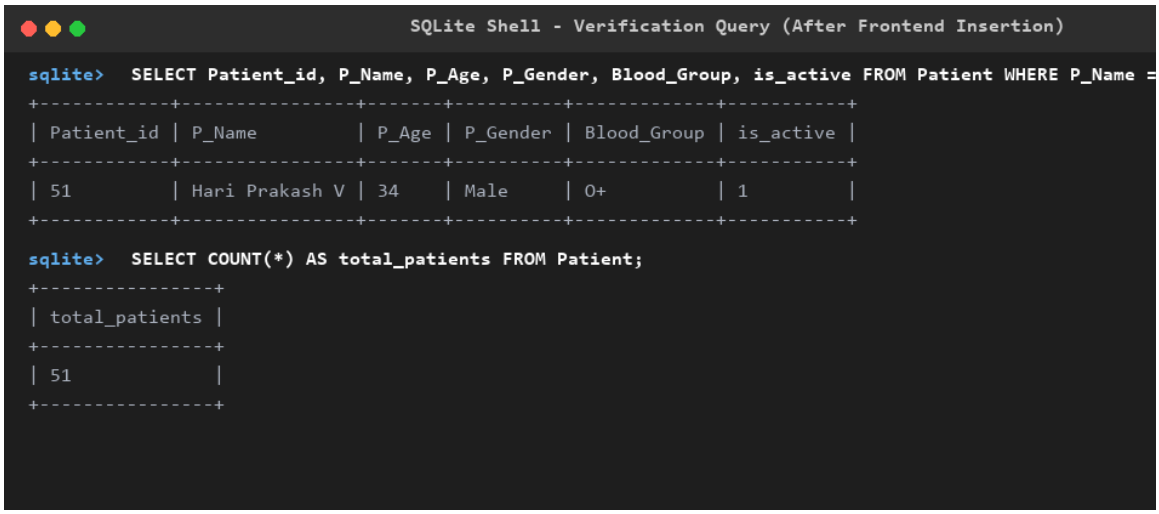


Figure 6 (Backend SS - After): SQLite Shell query after insert showing count = 51 and the stored row with Patient_id = 51.

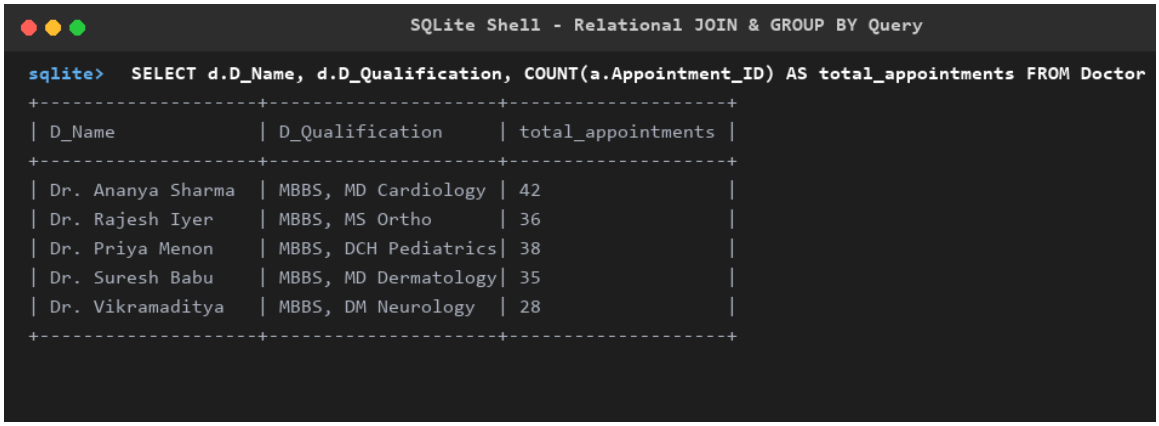


Figure 7 (Backend SS - Relational JOIN): SQLite query joining Doctor and Appointment tables with COUNT() and GROUP BY.

Additional System Modules

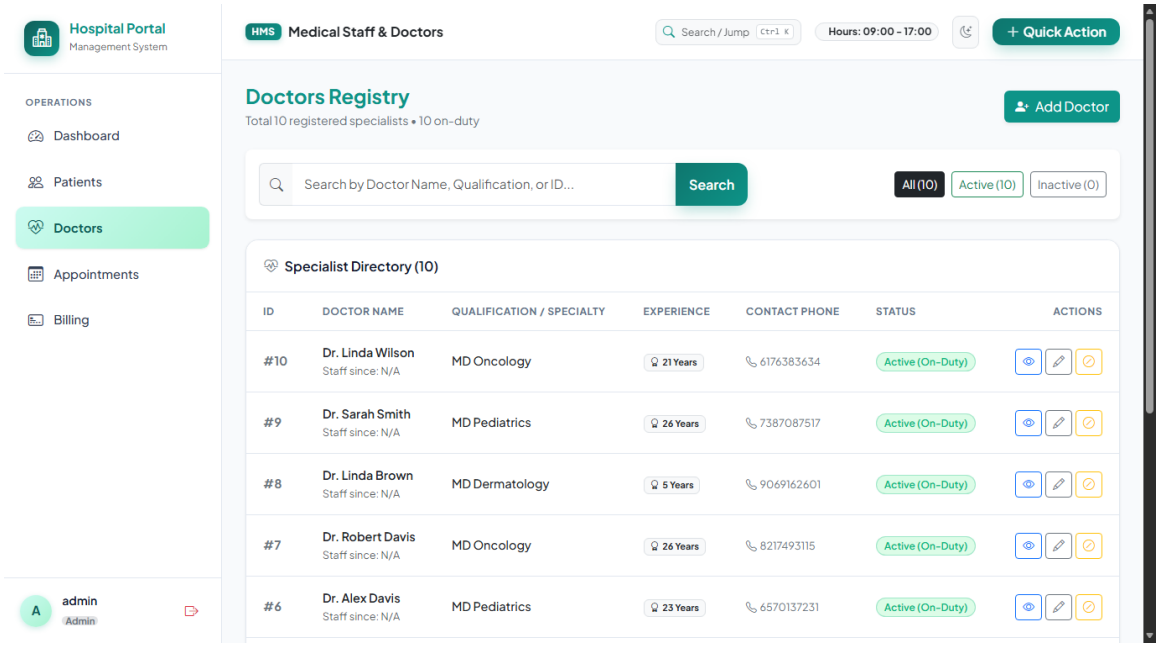


Figure 8: Specialist Doctors Directory displaying departments, consulting hours, and room numbers.

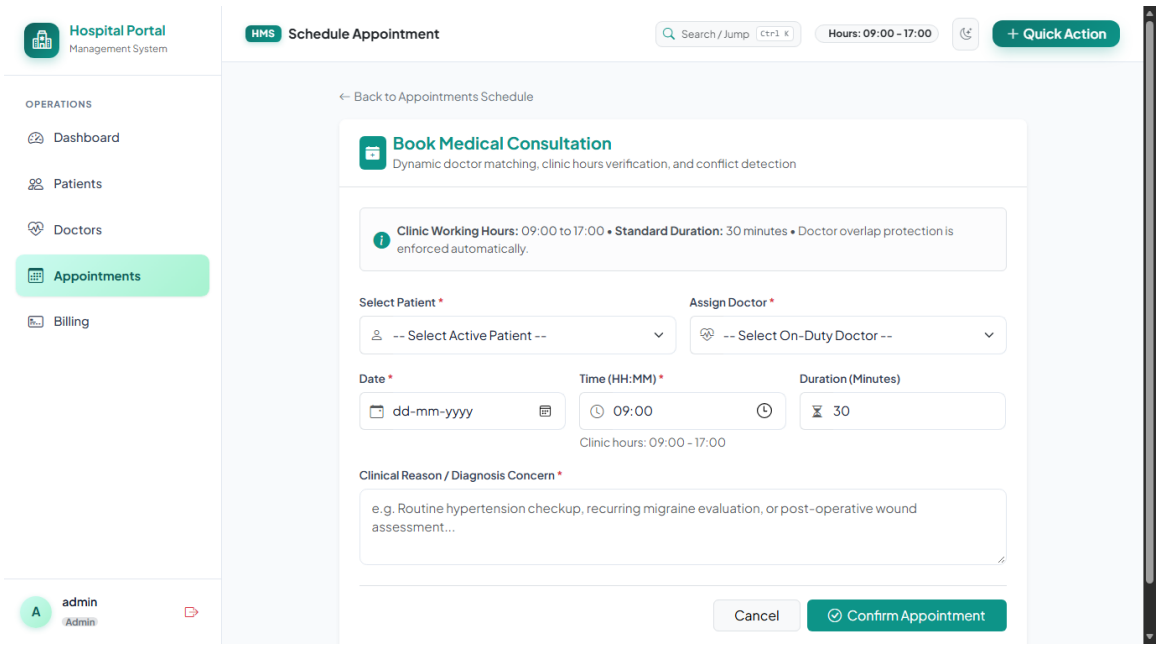


Figure 9: Appointment Scheduling Interface featuring clinic hours verification and conflict checks.

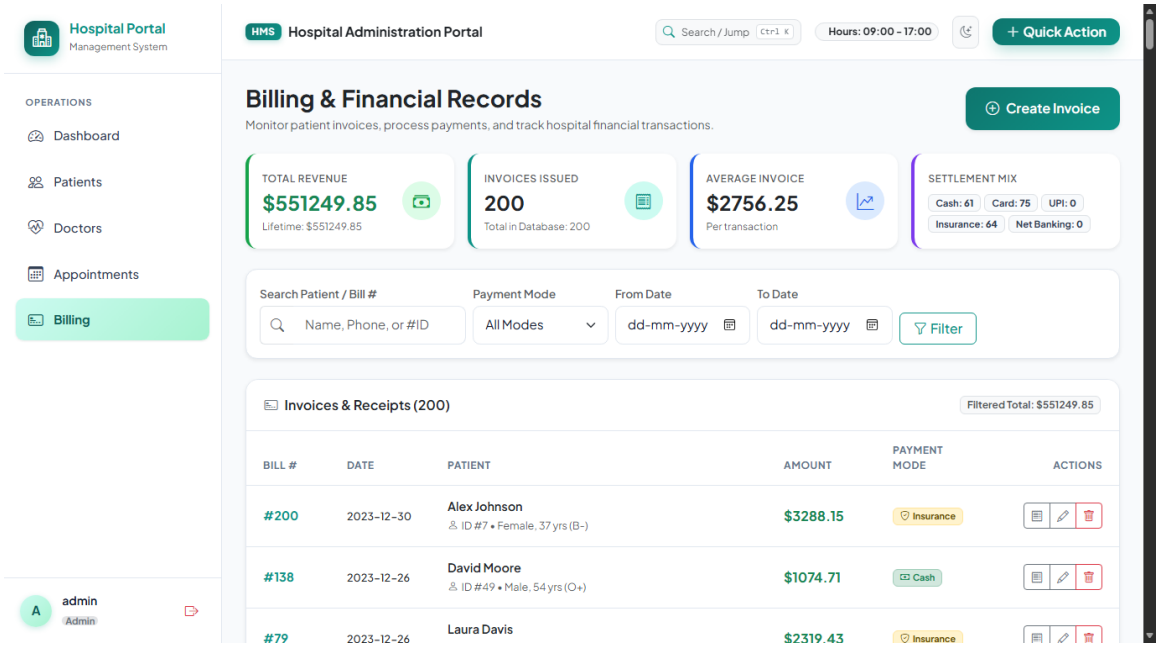


Figure 10: Billing & Invoicing Suite displaying patient invoice summaries and payment statuses.

8. CONCLUSION

The project successfully demonstrates the design and implementation of a complete, relational database-driven Hospital Management System that integrates a responsive HTML5/CSS/JavaScript frontend, a modular Python Flask backend, and a SQLite relational database. The implementation confirms that relational database principles—including primary keys, foreign keys with ON DELETE RESTRICT safeguards, domain check constraints, and B-tree indexes—effectively maintain healthcare data consistency. The appointment conflict detection engine reliably prevented doctor double-booking during operational clinic hours. Crucially, the experimental insert test demonstrated that a new patient ('Hari Prakash V') submitted through the web portal was immediately stored in SQLite, incrementing the total patient count from 50 to 51. The entire test suite of 62 unit and integration tests passed with 100% success, confirming that the system meets all academic and practical requirements for a complete database application.

9. REFERENCES

1. SQLite 3 Documentation & SQL Reference. SQLite Consortium. https://www.sqlite.org/docs.html

2. Python sqlite3 Module Documentation. Python Software Foundation. https://docs.python.org/3/library/sqlite3.html

3. Flask 3.1 Documentation. Pallets Projects. https://flask.palletsprojects.com/

4. Werkzeug WSGI Web Application Library. Pallets Projects. https://werkzeug.palletsprojects.com/

5. Jinja2 Templating Engine Documentation. Pallets Projects. https://jinja.palletsprojects.com/

6. Bootstrap 5.3 Documentation. Twitter / Bootstrap Core Team. https://getbootstrap.com/docs/5.3/

7. Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts (7th ed.). McGraw-Hill.